

Session 3 - Lab Programs

Name: H ASRAAR AHMED

Course name: DAA

Reg no: 192524028

Course code: CSA0604

Program 4

Question

Given an $m \times n$ grid and a ball at a starting cell, find the number of ways to move the ball out of the grid boundary in exactly N steps.

Example:

Input: $m=2, n=2, N=2, i=0, j=0$ -> Output: 6

Input: $m=1, n=3, N=3, i=0, j=1$ -> Output: 12

Program Code

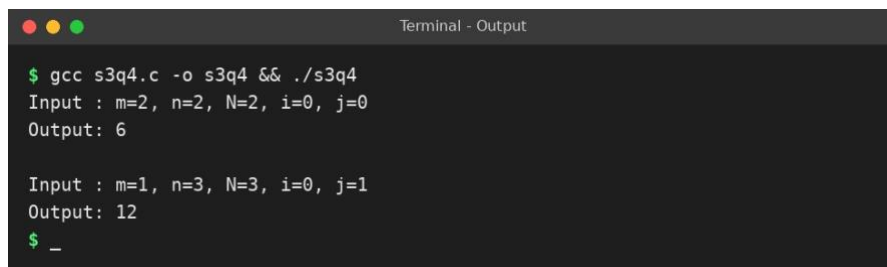
```
#include <stdio.h>
#include <string.h>

#define MOD 1000000007LL

/* dp[step][r][c] = number of ways to be at cell (r,c) after 'step' moves.
   Every move that leaves the grid is counted as one way to go out.   Time
   O(N*m*n), Space O(N*m*n) */ long long outOfGrid(int m, int n, int N, int si, int
sj) {    static long long dp[51][50][50];    memset(dp, 0, sizeof(dp));    int
dr[4] = {-1, 1, 0, 0}, dc[4] = {0, 0, -1, 1};    long long ways = 0;
dp[0][si][sj] = 1;    for (int s = 0; s < N; s++)        for (int r = 0; r < m;
r++)            for (int c = 0; c < n; c++) {                if (dp[s][r][c] ==
0) continue;                for (int d = 0; d < 4; d++) {                    int
nr = r + dr[d], nc = c + dc[d];                    if (nr < 0 || nr >= m || nc < 0
|| nc >= n)                        ways = (ways + dp[s][r][c]) % MOD;                /*
ball is out */                else                    dp[s + 1][nr][nc] =
(dp[s + 1][nr][nc] + dp[s][r][c]) % MOD;                }                return
ways;    }

int main() {    printf("Input : m=2, n=2, N=2, i=0, j=0\nOutput: %lld\n\n", outOfGrid(2,
2, 2, 0, 0));    printf("Input : m=1, n=3, N=3, i=0, j=1\nOutput: %lld\n", outOfGrid(1,
3, 3, 0, 1));    return 0; }
```

Output Screenshot



```
Terminal - Output

$ gcc s3q4.c -o s3q4 && ./s3q4
Input : m=2, n=2, N=2, i=0, j=0
Output: 6

Input : m=1, n=3, N=3, i=0, j=1
Output: 12
$ _
```

You are a professional robber planning to rob houses along a street. Each house has a certain amount of money stashed. All houses at this place are arranged in a circle. That means the first house is the neighbor of the last one. Meanwhile, adjacent houses have security systems connected, and it will automatically contact the police if two adjacent houses were broken into on the same night.

Examples:

Program 5

Question

(i) *Input: nums = [2, 3, 2] -> Output: The maximum money you can rob without alerting the police is 3 (robbing house 1).*

(ii) *Input: nums = [1, 2, 3, 1] -> Output: The maximum money you can rob without alerting the police is 4 (robbing house 1 and house 3).*

Program Code

```
#include <stdio.h>

int max(int a, int b) { return a > b ? a : b; }

/* Linear House Robber on nums[lo..hi]: Time O(n), Space O(1) */ int robLinear(int
nums[], int lo, int hi) {    int prev = 0, curr = 0;                /* best up to
previous / current house */    for (int i = lo; i <= hi; i++) {        int next =
max(curr, prev + nums[i]);        prev = curr;        curr = next;    }
return curr; }

/* Houses are in a circle: first and last are neighbours, so either skip
the last house or skip the first house and take the better result. */ int
robCircle(int nums[], int n) {    if (n == 1) return nums[0];    return
max(robLinear(nums, 0, n - 2), robLinear(nums, 1, n - 1)); }

void run(int nums[], int n) {    printf("Input : nums = [");    for (int i = 0; i < n;
i++) printf("%d%s", nums[i], i < n - 1 ? ", " : "");    printf("]\nOutput: The maximum
money you can rob without alerting the police is %d\n\n",        robCircle(nums, n));
}

int main() {    int a[] =
{2, 3, 2};    int b[] =
{1, 2, 3, 1};    run(a,
3);    run(b, 4);
return 0; }
```

Output Screenshot

You are climbing a staircase. It takes n steps to reach the top. Each time you can either climb 1 or 2 steps. In how many distinct ways can you climb to the top?

Examples:

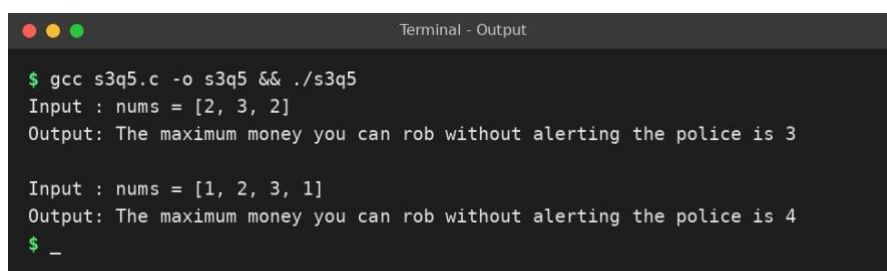
Input: $n=4$ -> Output: 5

Input: $n=3$ -> Output: 3

Program Code

```
#include <stdio.h>

/* ways(n) = ways(n-1) + ways(n-2) (last step was 1 or 2 stairs)
Time O(n), Space O(1) */ int climbStairs(int n) {    if (n <= 2)
return n;    int a = 1, b = 2;    /* ways for n = 1
and n = 2 */    for (int i = 3; i <= n; i++) {        int c = a
+ b;        a = b;        b = c;    }    return b; }
```



```
Terminal - Output

$ gcc s3q5.c -o s3q5 && ./s3q5
Input : nums = [2, 3, 2]
Output: The maximum money you can rob without alerting the police is 3

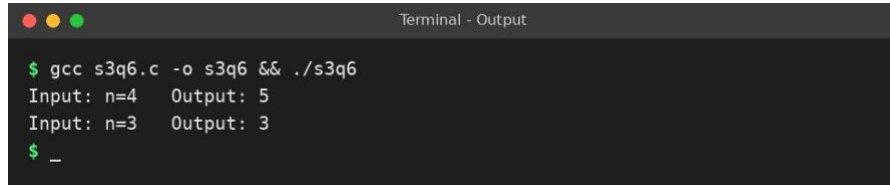
Input : nums = [1, 2, 3, 1]
Output: The maximum money you can rob without alerting the police is 4
$ _
```

Program 6

Question

```
int main() {    printf("Input: n=4    Output: %d\n",
climbStairs(4));    printf("Input: n=3    Output:
%d\n", climbStairs(3));    return 0; }
```

Output Screenshot



```
Terminal - Output

$ gcc s3q6.c -o s3q6 && ./s3q6
Input: n=4    Output: 5
Input: n=3    Output: 3
$ _
```

A robot is located at the top-left corner of a $m \times n$ grid. The robot can only move either down or right at any point in time. The robot is trying to reach the bottom-right corner of the grid. How many possible unique paths are there?

Examples:

Input: $m=7, n=3$ -> Output: 28

Input: $m=3, n=2$ -> Output: 3

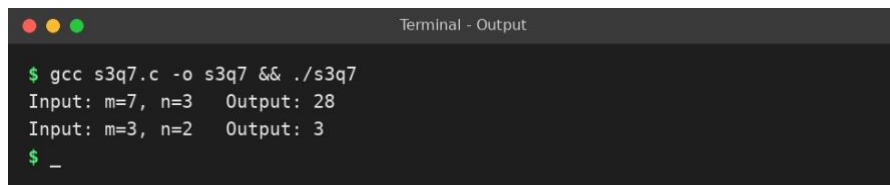
Program Code

```
#include <stdio.h>

/* dp[j] = number of paths to reach column j of the current row.    dp[j] +=
dp[j-1] (from above + from the left)    Time  $O(m \times n)$ , Space  $O(n)$  */ int
uniquePaths(int m, int n) {    int dp[100];    for (int j = 0; j < n; j++)
dp[j] = 1;    /* first row: only one way */    for (int i = 1; i < m;
i++)    for (int j = 1; j < n; j++)    dp[j] += dp[j - 1];
return dp[n - 1]; }
```

```
int main() {    printf("Input: m=7, n=3    Output: %d\n",
uniquePaths(7, 3));    printf("Input: m=3, n=2    Output:
%d\n", uniquePaths(3, 2));    return 0; }
```

Output Screenshot



```
Terminal - Output

$ gcc s3q7.c -o s3q7 && ./s3q7
Input: m=7, n=3    Output: 28
Input: m=3, n=2    Output: 3
$ _
```